

Dual-Model Crypto Trading System: Architecture & Implementation Plan

Executive Summary

This report outlines a complete plan to implement the dual-model (HMM + neural network) **signal-generation system** for cryptocurrency trading. We start by mapping the **original algorithms** (from the referenced equity paper ¹ ²) to crypto markets – discussing inputs, outputs, parameters, and assumptions for digital assets (24/7 trading, high volatility). We then propose a **system-level architecture**: data ingestion (exchange APIs, order books, on-chain data) feeding pre-processing, the HMM and neural modules, a signal combiner, a portfolio optimizer, and an execution engine. This includes data flows (WebSockets, REST) and interfaces (e.g. Binance API ³, Etherscan for blockchain data ⁴).

We detail the **hardware/software stack**: options such as cloud servers with GPUs, edge devices (e.g. NVIDIA Jetson ⁵) or FPGAs ⁶ for low-latency inference. Trade-offs (latency vs cost vs development effort) are discussed with example costs. We then cover **model adaptations**: feature engineering (crypto returns, log-returns, volatility, orderbook and on-chain metrics, sentiment), HMM state design (regime count, e.g. 4-state bull/bear/calm ¹), and neural architectures (LSTM, CNN, Transformer, hybrid conv-LSTM ²). We suggest training regimes and ensemble logic (e.g. weighted signals beyond simple agree/flat).

Block diagrams and Mermaid sequence charts illustrate component interactions and signal flow (data → models → signals → orders). An **implementation roadmap** with milestones (data pipeline, model training, integration, testing) and a CI/CD plan is provided. We propose **testing and validation**: using historical crypto data (exchange, on-chain) with backtesting frameworks, paper-trading, and metrics (Sharpe, Sortino, drawdown, latency, fill rates). Finally, we address **risk, security and regulatory** issues: exchange counterparty risk, custody, KYC/AML rules, market manipulation, and new South African rules (crypto under exchange control ⁷). Tables compare alternative hardware/software options and include a sample BOM with costs. Open assumptions (target assets, timeframe, budget, latency, leverage) are explicitly noted. All claims are supported by recent sources ¹ ² ³ ⁴ ⁷.

Original Dual-Model Algorithm (Equities) and Crypto Mapping

The reference paper's **dual-model strategy** uses a Hidden Markov Model (HMM) and a neural network to generate buy/flat signals. We recap it briefly and then discuss crypto-specific adaptation:

- **HMM (Regime Detection)**: The original uses a Gaussian HMM with *five states* to model market regimes from historical returns ¹. For crypto, we replace equities returns with **digital asset returns** (log-returns of price). Prior work shows a *4-state non-homogeneous HMM* often works best for Bitcoin/Ethereum ¹, distinguishing bull, bear, and neutral regimes. States should capture crypto volatility regimes (e.g. "rally", "crash", "quiet"). The model's input is typically daily log-returns (or higher-frequency returns) and it outputs a **state probability** or discrete regime label.

- **Neural Network (Forecasting):** The original used a feedforward net on price sequences. For crypto, time-series often benefit from sequence models: e.g. LSTM/GRU, 1D-CNN, or Transformer architectures. The referenced crypto DL review found a **Convolutional LSTM** (CNN+LSTM) performed best for multi-step price forecasting ². Other work highlights RNNs (LSTM/GRU) and CNNs for price trends. Inputs can include recent price windows, volume, volatility, and exogenous features. The NN outputs a predicted price or directional signal.
- **Signal Combination:** In the original, a *buy signal* was only issued if both models agreed on an up move ⁸. For crypto we can adopt or extend this: e.g. require HMM in bullish state *and* NN prediction > current price by threshold. More complex rules (weighted voting, meta-classifier) could be applied. We plan to experiment with ensemble rules (e.g. soft scores) rather than strict “both must agree”.
- **Outputs:** The system outputs periodic trading signals (e.g. end-of-day/interval buy or hold) and target allocations. These feed a portfolio optimizer (e.g. Black-Litterman) and execution. For crypto, shorting or using derivatives (futures) can be considered if allowed, though our baseline is **long-only** positions in chosen crypto assets.
- **Parameters/Assumptions:** Key parameters include lookback window size, HMM state count (e.g. 3–5), NN architecture and hyperparameters (layers, learning rate), and risk constraints. We assume **no fixed ETFs** but trade spot coins and possibly derivatives. Model assumptions such as log-normal returns may need revisiting for crypto’s fat tails. We treat results from equities as a starting point, but will tune to crypto market dynamics (24/7 operation, high volatility spikes ⁹).

System Architecture for Crypto

We propose a modular architecture tailored to cryptocurrency markets. Key components include:

- **Data Ingestion:**
 - *Exchange Data:* Real-time feeds from crypto exchanges (e.g. Binance, Coinbase, Kraken) via REST and WebSocket APIs. These provide order books, trades, and prices. For example, Binance’s API returns timestamps in milliseconds by default (microseconds with special header) ³.
 - *On-Chain Data:* Blockchain data (transaction counts, fees, chain metrics) via providers like Etherscan or full nodes. Etherscan’s API allows ~3 calls/sec (100k/day) on the free tier ⁴, suitable for on-chain indicators (gas price, transfers, address activity).
 - *Data Preprocessing:* Real-time data is parsed, cleaned (e.g. align times, handle missing points), and features computed. This includes computing log-returns, volatility, orderbook pressure, and on-chain signals. A **message queue** (e.g. Kafka, Redis) can buffer and stream data to models.
- **Strategy Engine (Dual Models):**
 - *HMM Module:* Inference engine that takes recent return series and computes current regime probabilities. Could be implemented in Python (hmmlearn) or optimized C++/GPU.
 - *Neural Module:* Inference engine for the NN (e.g. LSTM/CNN) that takes feature windows and outputs price forecasts or signals. We may train on GPUs and deploy inference on CPU/GPU (or edge).

- **Signal Combiner:** Logic that fuses HMM and NN outputs into trade signals. For example: if HMM indicates bullish regime *and* NN forecasts higher price, output “buy” else “flat”. We will explore weighted/thresholded combinations and machine learning meta-models for ensemble. ¹ ²
- **Portfolio Management:** A module (e.g. Black–Litterman or mean-variance optimizer) that allocates capital across coins based on signals, risk limits, and liquidity. Crypto-specific factors (token liquidity, stablecoin usage) are integrated here.
- **Execution Engine:** Connects to exchange **trading APIs** to place orders. It handles order types (market/limit), rate limits, and acknowledgments. For example, Binance’s REST API has 10s timeout for matching engine responses ³ ; our engine must handle timeouts and confirm fills via user-data streams or query calls.
- **Risk & Safety:** Monitors exposures, leverage (if using margin), and emergency stops. Since many crypto exchanges allow leverage, strict checks are needed. The default risk model may be minimal, but we will incorporate controls (max position sizes, stop-loss, cooling-off on drawdown).
- **Monitoring & Logging:** All data events, model decisions, orders, and trade results are logged for backtesting and compliance. Tools like Grafana/Prometheus can monitor system health and model drift.

⁵ Figure: Example edge computing platform (NVIDIA Jetson Nano) integrating CPU+GPU for on-device inference. The Jetson Nano Developer Kit is a compact “AI computer” for prototyping deep learning applications

```

flowchart LR
    ExchangeData[Exchange API / Order Book] --> Preprocess[(Data Preprocessing)]
    OnChainData[Blockchain / On-Chain API] --> Preprocess
    Preprocess --> HMMModel[HMM Model]
    Preprocess --> NNModel[Neural Model]
    HMMModel --> Combiner[Signal Combiner]
    NNModel --> Combiner
    Combiner --> PortfolioOpt[Portfolio Optimizer]
    PortfolioOpt --> Executor[Execution Engine]
    Executor --> Exchange[Exchange (Orders)]

```

Figure: High-level flowchart of data and signal flow in the crypto trading system.

```

sequenceDiagram
    participant Feed as DataFeed
    participant HMM as HMM_Module
    participant NN as NeuralNet_Module
    participant Comb as Combiner
    participant Opt as PortfolioOpt
    participant Ex as ExchangeAPI

```

```

Feed->>HMM: Send latest returns & features
Feed->>NN: Send latest price window & features
HMM->>Comb: Output regime prediction
NN->>Comb: Output price/momentum forecast
Comb->>Opt: Generate buy/hold/flat signal
Opt->>Ex: Place orders via exchange API
Ex->>Opt: Order confirmation/fills
Opt->>Comb: Trade execution status (fill, partial)

```

Figure: Sequence of operations from data receipt to order execution.

Hardware and Software Platform

Computing Infrastructure:

- **Cloud Server:** A dedicated server (e.g. AWS EC2, Google Cloud) with multi-core CPU and optional GPU (e.g. NVIDIA T4) is ideal for training and high-throughput backtesting. Cloud offers scalability and managed services (e.g. AWS Lambda, managed Kafka). However, it incurs ongoing costs and has network latency.
- **On-Prem/Edge Devices:** For lower-latency or continuous operation, we can use edge devices. Options include single-board computers like Raspberry Pi or NVIDIA Jetson Nano/Xavier. The Jetson Nano (quad-core ARM + 128-core GPU) provides ~472 GFLOPS for AI tasks ⁵ and costs ~\$100. Its successor Xavier (~\$400) offers more performance. These run Linux and support PyTorch/TensorRT for inference.
- **FPGAs:** For ultra-low latency (sub-ms), FPGAs (e.g. Xilinx Zynq or Intel Agilex) can implement HMM and neural inference in hardware ⁶. FPGAs give deterministic timing and efficiency but require specialized development (Verilog/HLS) and higher cost (\$200–1000+). They are best for latency-critical execution (e.g. high-frequency arbitrage), which is not our primary use case.
- **MCUs and DSPs:** Microcontrollers (STM32, TI) or DSP chips are low-power but typically too weak for deep learning. They may handle pre-processing or simple rule-based signals but not complex models.

Hardware Options (comparison):

Component	Example	Pros	Cons	Cost (est)
Cloud VM	AWS EC2/ GCP	Scalable, powerful GPUs, managed services	Latency to exchanges, ongoing fees	~\$0.50–\$3/hr
Workstation	Intel i9 + GPU	Max performance for training and backtests	High upfront cost, power usage	~\$1500–\$3000
NVIDIA Jetson	Nano, Xavier	On-device GPU for AI, low latency inference ⁵	Moderate cost, limited memory	Nano ~\$100, Xavier ~\$400
Raspberry Pi 4	Pi4	Very low cost, runs Linux/ Python	Limited CPU, no GPU, slower NN	~\$50–\$100

Component	Example	Pros	Cons	Cost (est)
FPGA (SoC)	Xilinx Zynq	Hardware acceleration, deterministic latency ⁶	Complex development, cost	~\$200–\$1000

Software Stack:

- *Operating System:* Ubuntu/Linux for servers and SoCs; embedded Linux or RTOS for real-time devices.
- *Languages:* Python (for model prototyping, data handling), C++ (for performance-critical inference). We will leverage open-source ML libraries.
- *ML Frameworks:* PyTorch or TensorFlow for NN training. For HMM, use libraries like `hmmlearn` or custom implementations. Models can be exported to ONNX for cross-platform inference (Jetson uses TensorRT).
- *Data Storage:* TimescaleDB/PostgreSQL or InfluxDB for storing historical price and on-chain data. Redis/Kafka for real-time streaming.
- *Messaging & APIs:* Use WebSockets/REST to exchanges (e.g. CCXT library for unified API access). For high throughput, consider FIX protocol or native SDKs.
- *Backtesting Framework:* Tools like Backtrader, Backtesting.py or Zipline can be adapted for crypto data. These will run on the same environment (preferably Linux, Python).
- *CI/CD:* Use GitHub Actions or Jenkins to automate code tests. Containerize services (Docker) for portability.

Trade-offs: Cloud servers maximize development speed and power but cost more over time and suffer network delays (hundreds of ms). Jetson-like devices reduce latency (<10 ms), lower cost, but require local management. FPGAs provide the fastest processing, but for crypto signals (not true high-frequency), FPGAs are likely overkill.

Model Design & Feature Engineering

Features: Beyond raw price, useful inputs include:

- **Returns/Price:** Log-returns, moving averages, momentum indicators (RSI, MACD).
- **Volume/Liquidity:** Trade volume, order book depth imbalance (bid/ask pressure), bid-ask spread.
- **Volatility:** Realized volatility (e.g. rolling standard deviation), VIX-equivalents (e.g. BTC volatility indexes).
- **On-Chain Metrics:** Transaction count, active addresses, hash rate, gas prices, stablecoin flow (Tether supply changes). These capture network usage and sentiment.
- **Sentiment/News (optional):** Tweets or news sentiment scores (requires NLP pipeline).

We will normalize features (z-scores) and align on time intervals (e.g. 1-minute or 1-hour bars). Because crypto trades 24/7, we must handle continuous time (no closing).

HMM Adaptation:

- *States:* Experiment with 3–6 states. Prior work found 4 states effective ¹. We anticipate states like “bull run”, “crash”, “consolidation”, “high volatility”.

- *Inputs*: Use log-returns or returns of several coins jointly (multivariate HMM). Crypto's fat tails may require robust fitting (t-distributions vs Gaussian).
- *Training*: Train on historical segments (e.g. last 2 years) and periodically retrain monthly as markets evolve.
- *Output*: State probabilities or Viterbi-decoded state. A state is classified as "positive" if historical mean return in that state is positive (and vice versa).

Neural Network Adaptations:

- *Architectures*: We will test RNNs and CNNs. Based on [46], a **multivariate Conv-LSTM** (1D convolution over time windows feeding LSTM) gave strong results ². We will also try pure LSTM/GRU and Transformer models (Time-series Transformer) to capture long dependencies.
- *Inputs*: Sequences of feature vectors (e.g. past 30 bars of price, volume, on-chain metrics). Use sliding windows to create training samples.
- *Hyperparameters*: Learning rate (~0.001), batch size, epochs (20-50), dropout (~0.1-0.3). We will use validation splits or cross-validation.
- *Target*: Predict next price or next return. Alternatively classify next-period up/down. Loss can be MSE or cross-entropy (for classification).
- *Multivariate vs Univariate*: Prior work shows **multivariate models (using multiple input features/coins) outperform univariate** ¹⁰, so we will include relevant co-features.
- *Ensemble Rules*: Instead of simple "both models must agree," we may compute a weighted signal: e.g. HMM-probability of up *and* NN-probability, and threshold that. We can also train a small meta-learner (logistic regression) to combine model outputs into a final signal.

Example Mermaid (model block):

```

graph LR
    PriceFeatures -->|windows| NeuralNet
    ReturnsSeries --> HMM
    NeuralNet --> SignalCombiner
    HMM --> SignalCombiner
    SignalCombiner --> TradingSignal

```

Implementation Roadmap

A phased development timeline (0-indexed in weeks):

1. **Setup & Data (Wks 0-2):**
2. Secure historical crypto datasets (e.g. via CCXT for OHLCV data, Kaggle datasets, Blockchain APIs).
3. Build ETL pipeline: ingest from exchanges (e.g. Binance spot historical, Coinbase, etc.) and on-chain (Etherscan or node). Normalize and store in DB.
4. Set up development environment (Python, libraries, version control, Docker containers).

5. Model Prototyping (Wks 3–6):

6. Implement HMM (e.g. using `hmmlearn` or custom) and test on 1-year BTC data. Optimize state count (e.g. compare 3–6 states via BIC).
7. Build NN models (LSTM, CNN) in PyTorch. Conduct experiments for window size, features. Evaluate on train/test (e.g. train on 2018–2021, test on 2022–2023).
8. Combine signals: define simple threshold rule (both must be up) and test on historical data.
9. Use GPU instance (cloud or local) for training. Track performance (loss, backtest results).

10. Integration & Backtesting (Wks 7–9):

11. Integrate models into a trading loop within a backtesting framework. Automate training and inference pipeline.
12. Backtest the strategy over 2018–2024 for multiple coins. Compute metrics (cumulative return, Sharpe, max DD). Adjust hyperparams.
13. Paper-trade in a sandbox or demo account (exchanges often provide testnet or simulation modes).

14. Deployment & Interfaces (Wks 10–12):

15. Develop the real-time system: set up a server/VM or edge device. Configure continuous data feed (WebSockets).
16. Containerize components: model inference service (TensorRT for Jetson, or ONNX), order execution service, data collector.
17. Integrate exchange APIs for orders: e.g. use Binance's REST/WebSocket for price and orders. Ensure API keys and secrets are secured.
18. Implement CI/CD: use GitHub Actions to test code on push, Docker to deploy images, monitoring alerts on failures.

19. Testing & Refinement (Wks 13–15):

20. Perform stress tests: simulate high-frequency data bursts, ensure no queue overflows.
21. Latency measurement: end-to-end (data receipt → model inference → order placement). If too slow, optimize bottlenecks (e.g. run NN on GPU).
22. Add logging/alerts: e.g. alert if no data received or models error, or if positions go out-of-bounds.
23. Retraining schedule: set up automated re-training (e.g. weekly) with latest data, and model versioning.

24. Live Launch Preparation (Wks 16+):

25. Final risk checks: start with small capital and limited position sizes.

26. Monitor performance metrics in real trading vs backtest (drift detection).

Milestones:

- M1: Data pipeline validated (raw data -> features).
- M2: HMM and NN standalone models working (unit tests passed).
- M3: Backtest results meet target (e.g. Sharpe > 0.5 after fees).
- M4: Real-time trading demo (paper / small capital).
- M5: Full-scale live deployment.

Testing, Metrics & Verification

Datasets & Simulation:

- **Historical Data:** Use minute-resolution OHLCV data for major coins (BTC, ETH, others) from 2017 onward. Also collect order book snapshots (for volume/liquidity features) if possible. On-chain: historical snapshots of active addresses, transaction volumes. Sources: CryptoCompare, Kaiko, exchange CSV dumps, or free APIs (e.g. Binance, CoinGecko).
- **Backtesting:** Use frameworks like Backtrader or custom Python scripts to replay historical data. Include realistic constraints: exchange fees (0.1–0.2% spot), order slippage, and latency (simulate delay).
- **Paper-Trading:** Platforms like BitMEX Testnet or Binance Testnet can simulate order execution.

Simulation Tools:

- Python environments for backtest (Pandas, NumPy).
- Jupyter/MLflow for experiment tracking.
- Unit tests for model consistency (e.g. check HMM state transitions on synthetic data).

Performance Metrics:

- *Trading performance:* Annualized return, Sharpe ratio, Sortino ratio, max drawdown, win rate, profit factor. Aim for Sharpe > 0.5 after fees as a baseline.
- *Model accuracy:* For NN predictions, MSE or directional accuracy on hold-out sets.
- *Signal metrics:* Hit rate (percentage of signals that are correct in hindsight).
- *Execution metrics:* Latency (time from data receipt to order) <100ms for low-frequency strategy; order fill rate (percent of orders fully filled within expected slippage).
- *Reliability:* Uptime %, number of failures.

Acceptance Criteria:

- The strategy must outperform a naive benchmark (e.g. buy-and-hold of BTC) over test period.
- Latency and throughput must meet system requirements (e.g. handle data at required frequency without backlog).
- No critical failures in tests: e.g. under simulated exchange downtime, system fails safe (no orders).
- Models should generalize: no severe overfitting (check by cross-validating on different time periods).

Verification:

- *Cross-Validation*: Time-series cross-validation (rolling window) to ensure stability.
- *Sensitivity Analysis*: Vary key parameters (HMM states, NN window) and check robustness.
- *A/B Tests*: Compare dual-model vs single-model strategies.

Risk, Security, and Regulatory Considerations

- **Counterparty Risk**: Crypto exchanges can fail or be hacked. We will use reputable exchanges (e.g. Binance, Coinbase) and distribute funds (don't keep all assets on one exchange). Consider using institutional-grade custody (e.g. Fireblocks) in production.
- **Custody & Keys**: Private keys or API keys must be stored securely (hardware security modules or vaults). Use multi-factor auth for accounts.
- **KYC/AML Compliance**: If deploying in SA, follow FSCA guidance. Note South Africa's recent move to treat crypto under exchange control ⁷ means cross-border crypto flows may need SARB approval. Ensure all trading abides by KYC/AML laws (register as financial services provider if needed).
- **Market Manipulation**: Crypto markets are prone to pump-and-dumps. Include checks for abnormal price spikes. Avoid highly illiquid tokens.
- **Smart Contract Risk**: If using DeFi or lending protocols for execution, audit contracts. For spot trading, ensure exchange's orderbook API is trustworthy.
- **Cybersecurity**: Protect systems from intrusion. Use secure coding, rate-limit APIs. The usual OWASP/DevSecOps guidelines apply.
- **Legal**: Monitor regulations. In SA, exchanges currently must register with FSCA; upcoming crypto exchange control requires approval for offshore crypto transfers ⁷. Ensure any cross-border strategy accounts for such constraints.

Alternatives and Bill of Materials

We compare platforms, frameworks, and list a sample BOM:

Category	Option	Pros	Cons
Cloud VM	AWS EC2 (GPU)	Scalable, GPU support, managed infra	Pay-as-you-go costs, latency
Edge Device	NVIDIA Jetson Nano	Integrated GPU for NN, low-latency inference ⁵	Limited memory (4GB), moderate cost
Edge Device	Raspberry Pi 4	Very low cost, large community	No GPU, slower NN inference
FPGA/SoC	Xilinx Zynq Ultrascale	Ultra-parallel, determinism ⁶	High complexity, power use
Trading API	CCXT (Python)	Unified API for many exchanges	Extra abstraction layer overhead
DB	PostgreSQL/ Timescale	Robust, time-series support	Admin overhead

Category	Option	Pros	Cons
Messaging	Kafka/Redis	High-throughput, real-time streaming	Requires cluster management

Sample Bill of Materials (Startup Scale):

- Cloud compute (AWS EC2 g4dn.xlarge GPU, ~ \$0.80/hr) for 1-year dev/testing: ~\$7,000 (on-demand)
- Jetson Nano Developer Kit – \$100 ⁵
- 32 GB microSD (for Jetson) – \$15
- Ethernet cables, power supply – \$25
- Server (AWS or on-prem) – \$1,500 (one-time or amortized)
- Total ≈ \$8,000 for development setup (variable for cloud usage).
- **Note:** Actual trading capital is separate and depends on strategy.

Assumptions and Open Issues

- **Target Assets:** We assume top-cap cryptocurrencies (BTC, ETH, LTC, etc). Choice of assets is open; strategy should start with liquid majors.
- **Timeframe:** We target intraday signals (e.g. 1-hour or 15-min bars) for moderate-frequency trading. Ultra-high-frequency (ms) is out of scope.
- **Budget:** Implementation budget is open. We assume moderate R&D budget (thousands to tens of thousands USD) but not unlimited.
- **Latency:** Not explicitly given; assume millisecond to second-level latency is acceptable. If sub-ms needed, consider co-location (not planned here).
- **Leverage:** Strategy is **long-only** by default (no shorting). Enabling margin/leverage would add complexity (interest costs, liquidation risk).
- **Infrastructure:** We assume Internet connectivity is reliable. If deployed in edge (e.g. data center), network delays are minimal.
- **Regulatory:** Not explicitly given, but assume compliance with local laws (SA Exchange Control, KYC/AML) and no aim to trade on unregulated venues.

Open-ended issues (for client specification): exact latency tolerance, maximum capital, accepted risk limits, choice of exchanges (some restrict US customers), and whether to include decentralized (DEX) trading. These should be clarified in requirements.

Sources: We have drawn on the referenced dual-model paper and recent crypto ML literature ¹ ² ⁷, as well as official docs (Binance API ³, Etherscan ⁴) and hardware references ⁵ ⁶ to inform this design.

¹ Exploring the predictability of cryptocurrencies via Bayesian hidden Markov models - ScienceDirect
<https://www.sciencedirect.com/science/article/abs/pii/S0275531921001756>

² ⁹ ¹⁰ [2405.11431] Review of deep learning models for crypto price prediction: implementation and evaluation
<https://arxiv.org/abs/2405.11431>

3 General API Information | Binance Open Platform

<https://developers.binance.com/docs/binance-spot-api-docs/rest-api>

4 Etherscan APIs - Ethereum (ETH) API Provider

<https://etherscan.io/apis>

5 Get Started With Jetson Nano Developer Kit | NVIDIA Developer

<https://developer.nvidia.com/embedded/learn/get-started-jetson-nano-devkit>

6 Understanding FPGAs - From Architecture to SoC, RFSoc, and Embedded FPGA Technologies Logic Fruit Technologies

<https://www.logic-fruit.com/blog/fpga/understanding-fpgas/>

7 South Africa: Crypto Assets Likely to Enter Exchange Control Regime | Insight | Baker McKenzie

<https://www.bakermckenzie.com/en/insight/publications/2026/03/south-africa-crypto-assets-likely-to-enter-exchange-control-regime>

8 AI-Powered Energy Algorithmic Trading: Integrating Hidden Markov Models with Neural Networks by Tiago Monteiro :: SSRN

https://papers.ssrn.com/sol3/papers.cfm?abstract_id=5756885